Report: Speech-to-Text Module

(Automatic Speech Recognition - ASR)

AuraTranslate Edge - OneVoice AI Challenge 2026

Lê Gia Khánh - Gia Sư Đỉnh Cao

Contents

1 Giới thiệu **2**
1.1 Project AuraTranslate Edge . . . 2
1.2 Module Automatic Speech Recognition (ASR) . . . 2
1.3 GitHub mã nguồn . . . 2

2 Lựa chọn Models **2**
2.1 Thí nghiệm . . . 2
2.2 Kết luận . . . 3

3 Model Zipformer (Vi) **3**
3.1 Kiến trúc Model . . . 3
3.2 Quantize và Deploy . . . 5
3.3 Những khó khăn và Cách khắc phục . . . 6
3.4 Mã nguồn và Cách triển khai . . . 7
3.5 Kết quả từ Qualcomm AI Hub . . . 9

4 Model SenseVoice (En/Zh/Ko) **10**
4.1 Kiến trúc Model . . . 10
4.2 Quantize và Deploy . . . 12
4.3 Những khó khăn và Cách khắc phục . . . 13
4.4 Mã nguồn và Cách triển khai . . . 14
4.5 Kết quả từ Qualcomm AI Hub . . . 16

5 Demo ASR Dashboard **18**
5.1 Mã nguồn và Cách triển khai . . . 18
5.2 Nhận xét kết quả . . . 19

6 Pipeline Integration **19**
6.1 Các Modules còn lại . . . 19
6.2 Hướng hợp nhất Pipeline . . . 20

1 Giới thiệu

1.1 Project AuraTranslate Edge

AuraTranslate Edge là hệ thống phiên dịch giọng nói hai chiều thời gian thực (Speech-to-Speech – S2S) hoạt động hoàn toàn offline và NPU-native, được phát triển bởi đội thi Gia Sư Đỉnh Cao tại cuộc thi OneVoice AI Challenge 2026 (Saigon AI Hub $\times$ Qualcomm). Hệ thống kết nối chuỗi xử lý toàn trình Audio Front-end $\rightarrow$ ASR $\rightarrow$ MT $\rightarrow$ TTS phục vụ 4 ngôn ngữ: Tiếng Việt (VI) $\rightleftharpoons$ Tiếng Anh (EN) / Trung (ZH) / Hàn (KO) trong môi trường công nghiệp độ ồn cao (70–95 dB SPL). Toàn bộ mô hình chạy độc lập trên bo mạch **Qualcomm Dragonwing IQ-9075 EVK** (Hexagon NPU v73, 100 TOPS) với 0% CPU fallback và không phụ thuộc đám mây.

1.2 Module Automatic Speech Recognition (ASR)

Mô-đun ASR (Step 1) đảm nhận chuyển đổi sóng âm thành văn bản, sử dụng kiến trúc định tuyến 2 động cơ tối ưu hóa đồ thị tĩnh trên NPU:

- **Zipformer-150M-CR-CTC (Tiếng Việt):** Đóng gói thành đồ thị tĩnh duy nhất (Single Static DAG 4 khối), biên dịch Native FP16 trên Hexagon NPU. Mô hình đạt độ trễ 184.31 ms cho 15 giây âm thanh, RAM đỉnh 7.78 MB và xuất trực tiếp luồng byte UTF-8 ra bộ nhớ chia sẻ (Zero-CPU Detokenize).
- **SenseVoice-Small (Ngoại ngữ EN/ZH/KO):** Kiến trúc phi tự hồi quy (NAR) 50 lớp Transformer tích hợp sẵn nhận diện ngôn ngữ và chuẩn hóa văn bản, triển khai theo dạng Split-Graph W8A16 Mixed Precision. Mô hình đạt độ trễ 269.40 ms cho 29 giây âm thanh (RTF $= 0.0093$) với 100% toán tử chạy trên NPU.

1.3 GitHub mã nguồn

Mã nguồn dự án được công khai tại:
https://github.com/LGKAI/AuraTranslateEdge-OneVoice

2 Lựa chọn Models

2.1 Thí nghiệm

Nhóm tiến hành thực nghiệm đối đầu trên tập dữ liệu chuẩn FLEURS kết hợp tạp âm nhà xưởng ở 6 mức SNR (Sạch đến 0 dB), đánh giá qua WER (Tiếng Việt, Tiếng Anh), CER (Tiếng Trung, Tiếng Hàn) và hệ số thời gian thực (RTF).

Table 1: Đánh giá đối đầu nhánh Tiếng Việt qua 6 mức SNR

Mức SNR	PhoWhisper-small	Zipformer-30M	Moonshine-tiny	Qwen3-ASR-0.6B
Sạch	5.51%	**5.35%**	7.70%	5.90%
20 dB / 15 dB	5.51% / 6.05%	**5.89% / 5.89%**	7.10% / 8.20%	4.60% / 5.20%
10 dB	7.11%	**6.95%**	11.20%	7.00%
5 dB	13.47%	**6.22%**	26.10%	13.70%
0 dB (Cực ồn)	8.01%	**4.10%**	16.40%	6.60%
RTF TB	0.06–0.18	**0.017–0.05**	0.05–0.30	0.10–0.17

Table 2: Đánh giá đối đầu nhánh Ngoại ngữ ở điều kiện Sạch và Nhiễu nặng (0 dB)

Điều kiện	Ngôn ngữ / Chỉ số	SenseVoice-Small	Moonshine	Qwen3-ASR-0.6B
Sạch	Tiếng Anh (WER)	6.8%	9.6%	**4.9%**
	Tiếng Trung (CER)	**2.3%**	16.2%	9.1%
	Tiếng Hàn (CER)	4.5%	8.1%	**4.4%**
	RTF TB	**0.009–0.017**	0.05–0.30	0.10–0.17
SNR = 0 dB	Tiếng Anh (WER)	11.5%	25.1%	**7.0%**
	Tiếng Trung (CER)	11.8%	78.6%*	**10.2%**
	Tiếng Hàn (CER)	20.9%	37.0%	**16.1%**

(Ghi chú: Moonshine tại SNR 0 dB Tiếng Trung bị suy sập nhận dạng hoàn toàn).

2.2 Kết luận

Căn cứ kết quả thực nghiệm, dự án chốt giải pháp kiến trúc:

- **Chọn Zipformer (Tiếng Việt):** Kháng nhiễu vượt trội ở điều kiện khắc nghiệt (WER 6.22% tại 5 dB và 4.10% tại 0 dB, tốt hơn gấp đôi PhoWhisper); RTF siêu nhanh 0.017–0.05; mô hình siêu nhẹ (29.3 MB INT8) và hỗ trợ streaming tự nhiên qua khối Joiner.
- **Chọn SenseVoice-Small (Ngoại ngữ EN/ZH/KO):** Tốc độ giải mã nhanh nhất (RTF 0.009–0.017, nhanh gấp $10\times$ Qwen3); CER tiếng Trung 2.3% và tiếng Hàn 4.5%; tích hợp sẵn LID và ITN trong một lượt suy luận duy nhất.
- **Loại bỏ các ứng viên khác:** PhoWhisper suy giảm chất lượng mạnh khi có tiếng ồn; Moonshine suy sập ở môi trường nhiễu nặng; Qwen3-ASR có độ trễ quá lớn (0.10–0.17) làm cạn kiệt ngân sách thời gian thực của thiết bị biên.

3 Model Zipformer (Vi)

3.1 Kiến trúc Model

Để đạt mục tiêu 100% NPU-native và triệt tiêu hoàn toàn nghẽn cổ chai CPU Host (0% CPU fallback), dự án sử dụng biến thể **Zipformer-150M-CR-CTC** (huấn luyện trên 6.000 giờ tiếng Việt thực tế) và hợp nhất toàn bộ luồng xử lý thành một Đồ thị Tính toán Tĩnh duy nhất (**Single Static DAG**) gồm 4 khối chức năng liên hoàn:

Audio (15s) $\xrightarrow{\text{Khối 1: Fbank}}$ Khối 2: Encoder $\xrightarrow{\text{Khối 3: CTC Collapse}}$ Khối 4: Detokenizer $\rightarrow$ Chuỗi Byte UTF-8

1. Khối Fbank DSP Tĩnh (Matrix-based Audio Front-end): Toàn bộ thuật toán xử lý tín hiệu số (DSP) truyền thống được chuyển hóa thành các phép nhân ma trận thuần túy để chạy trực tiếp trên Hexagon Tensor Processor (HTP):

- *Phân khung song song (Framing):* Tín hiệu sóng âm $[1, 240240]$ (15 giây @ 16 kHz) được trượt qua lớp tích chập `Conv1D` với ma trận đơn vị $W_{\text{identity}} \in \mathbb{R}^{400\times1\times400}$ (stride $= 160$), trích xuất đồng thời 1.500 khung thoại (mỗi khung 400 mẫu, 25 ms) trong một bước tính mà không cần vòng lặp động.
- *Cửa sổ Povey & DFT ma trận:* Nhân từng phần tử với cửa sổ Povey $w[n]$ để chống rò rỉ phổ, sau đó tính biến đổi Fourier rời rạc qua tích vô hướng với hai ma trận hằng số hình sin/cos Thực và Ảo ($W_{\text{real}}, W_{\text{imag}} \in \mathbb{R}^{400\times201}$, $N_{\text{fft}} = 512$), quy toàn bộ khâu tính phổ về phép nhân ma trận (`MatMul`).
- *Bộ lọc Mel & Nén Logarithm:* Chiếu phổ năng lượng qua ma trận Mel $M_{\text{mel}} \in \mathbb{R}^{201\times80}$ và lấy log tự nhiên (ngưỡng sàn $\epsilon = 10^{-5}$), xuất ra ma trận đặc trưng chuẩn $Y_{\text{fbank}} \in \mathbb{R}^{1\times1500\times80}$.

2. Khối Zipformer Acoustic Encoder & CTC Projection Head: Bóc tách các đặc trưng ngữ âm sâu từ ma trận Fbank và dự đoán phân phối xác suất từ vựng:

- *Nén thời gian 4$\times$ (Conv2D Subsampling):* Hai lớp tích chập 2D bước nhảy 2 nén 1.500 khung xuống còn 375 khung ($T' = 375$, mỗi khung 40 ms). Việc giảm 4 lần chiều dài chuỗi giúp giảm tới $16\times$ khối lượng tính toán Attention ở các tầng sau ($\mathcal{O}(1500^2) \rightarrow \mathcal{O}(375^2)$), tiết kiệm lớn bộ nhớ đệm SRAM/VTCM.
- *Lõi Zipformer đa tỷ lệ (Multi-rate Stacks):* Kết hợp các nhánh xử lý ở nhiều độ phân giải thời gian khác nhau (*U-Net style*), bao gồm FFN kiểu Macaron, cơ chế Multi-Head Self-Attention (học ngữ cảnh toàn câu) và Depthwise Separable Convolution (bắt âm học cục bộ). Chiều ẩn đầu ra đạt $H_{\text{enc}} \in \mathbb{R}^{1\times375\times768}$.
- *Chuyển đổi RNN-T sang CTC Head để loại bỏ CPU Fallback:* Thay vì dùng Decoder/Joiner tự hồi quy bắt buộc phải chạy vòng lặp `while` trên CPU, nhóm thay thế bằng lớp *Linear CTC Projection* ($W_{\text{ctc}} \in \mathbb{R}^{768\times2000}$). Bước này quy toàn bộ mạng về phép nhân ma trận tĩnh, xuất thẳng ma trận xác suất 2.000 token trên 375 khung thời gian ($[1, 375, 2000]$) hoàn toàn trên NPU.

3. Khối Static CTC Collapse (Khử trùng & Lọc Blank trên Đồ thị Tĩnh): Thu gọn ma trận dự đoán 375 khung thành chuỗi token từ vựng thực tế mà không cần logic điều kiện của CPU:

- *Bản chất bài toán:* Người nói phát âm kéo dài qua các khung 40 ms khiến mô hình sinh ra nhiều token trùng lặp liên tiếp (ví dụ: `"học", "học"`), xen kẽ bởi ký tự rỗng (`<blank>`, ID = 0). Do NPU không hỗ trợ vòng lặp duyệt mảng hay mảng co giãn (`push_back`), thuật toán phải được ma trận hóa tĩnh:

  1. Tìm arg max song song trên 375 khung để lấy mảng nhãn có xác suất cao nhất $Y \in \mathbb{R}^{1\times375}$.

2. Áp dụng mặt nạ nhị phân (*Indicator Mask*): đánh dấu 1 tại vị trí token thay đổi so với khung liền trước và khác ký tự rỗng ($y_t \neq y_{t-1} \wedge y_t \neq 0$).

3. Dùng toán tử cộng dồn tiền tố `CumSum` kết hợp `ScatterElements` để dồn toàn bộ token hợp lệ về đầu tensor cố định $[1, 375]$, các vị trí còn lại đệm số 0.

4. Khối Zero-CPU Static Byte Detokenizer (Giải mã Byte Trực tiếp trên NPU): Xuất kết quả văn bản đọc được ra bộ nhớ chia sẻ mà không tốn tài nguyên máy chủ:

- *Nhúng bảng mã tĩnh vào bộ nhớ VTCM:* Toàn bộ từ điển 2.000 tokens BPE được số hóa thành ma trận byte UTF-8 tĩnh $M_{\text{byte}} \in \mathbb{R}^{2000\times 12}$ (mỗi hàng lưu chuỗi byte của một token) và nạp thẳng vào bộ nhớ đệm nội tại cực nhanh VTCM của Hexagon NPU.

- *Giải mã tức thời tại Host CPU (< 0.001 ms):* NPU dùng toán tử `Gather` tra cứu song song và xuất trực tiếp mảng số nguyên chứa các mã byte UTF-8 ra RAM. CPU Host chỉ cần gọi hàm nguyên bản của hệ điều hành `bytes.decode('utf-8')` để hiển thị văn bản kết quả, đạt tốc độ giải mã tức thời ($< 1\,\mu$s) và loại bỏ hoàn toàn các thư viện SentencePiece hay HuggingFace cồng kềnh.

3.2 Quantize và Deploy

Để hiện thực hóa triết lý vận hành 100% NPU-native trên thiết bị biên, toàn bộ chu trình ASR tiếng Việt được tích hợp thành một Đồ thị Tính toán Tĩnh duy nhất (**Single Static DAG**) thông qua công cụ `onnx.compose.merge_models`. Chuỗi xử lý kết nối liên hoàn 4 phân hệ: Fbank DSP $\rightarrow$ Zipformer Encoder $\rightarrow$ CTC Collapse $\rightarrow$ UTF-8 Detokenize thành một tệp ONNX độc lập, loại bỏ hoàn toàn các điểm giao tiếp trung gian với CPU Host.

1. Phân tích Thách thức Toán học của RNN-T và Quyết định Chuyển đổi CTC Head:

- *Bản chất vòng lặp động của RNN-T gốc:* Mô hình nguyên bản *Zipformer-30M-RNNT* giải mã chùm tia (Beam Search) theo hai chiều thời gian và nhãn (t, u) qua vòng lặp `while` động: tại mỗi khung t, hệ thống liên tục dự đoán nhãn $u + 1$ cho đến khi gặp ký tự rỗng (`<blank>`) mới chuyển sang khung $t + 1$.

- *Rào cản trên phần cứng NPU:* Qualcomm Hexagon HTP hoạt động theo nguyên lý đồ thị tĩnh (Static Dataflow) và không hỗ trợ cấu trúc rẽ nhánh điều kiện động. Nếu giữ nguyên RNN-T, mỗi bước lặp sinh token đều phải đẩy về CPU Host xử lý (CPU fallback), khiến hệ số RTF tăng vọt từ 0.01 lên > 0.5.

- *Giải pháp kiến trúc không cần huấn luyện lại (No Retraining):* Nhóm chuyển sang checkpoint **Zipformer-150M-CR-CTC-RNNT-6000h** (huấn luyện đa mục tiêu trên 6.000 giờ tiếng Việt) có sẵn nhánh *Linear CTC Head* độc lập ($W_{\text{ctc}} \in \mathbb{R}^{768\times 2000}$, $b_{\text{ctc}} \in \mathbb{R}^{2000}$). Nhờ giả định tính độc lập có điều kiện giữa các khung thoại của thuật toán CTC:

$$P(Y|X) = \prod_{t=1}^{T} P(y_t|X)$$

toàn bộ khối Decoder và Joiner được loại bỏ. Quy trình giải mã quy về phép nhân ma trận tĩnh chiếu song song trên 375 khung thời gian, giúp mô hình chạy thẳng một mạch trên NPU mà không cần CPU can thiệp.

2. Quá trình Lượng tử hóa và Biên dịch Phần cứng (Từ W8A16 đến Native FP16):

- *Sự cố cắt cụt dải động ở chuẩn W8A16 ban đầu:* Phổ năng lượng DSP sau hàm log mang giá trị âm nhỏ trong khoảng $[-11.5, 3.2]$, trong khi các tầng sâu của Zipformer có dải kích hoạt rộng $[-50, 50]$. Thuật toán lượng tử hóa sau huấn luyện (PTQ) chuẩn INT8 chỉ có 256 mức đã tự động giãn thang đo bao trùm toàn dải cực trị, khiến biên độ nhỏ ở những khung đầu câu bị làm tròn về 0. Hậu quả là CTC Head dự đoán nhầm thành token rỗng, gây hiện tượng mô hình nuốt mất từ đầu tiên của câu thoại.

- *Nâng cấp W16A16 và Biên dịch Native FP16 DLC:* Nhóm nâng cấu hình kích hoạt lên 16-bit (W16A16, cung cấp 65.536 mức rời rạc) và biên dịch Native FP16 trên Hexagon NPU v73 với cờ tối ưu:

  ```
  --target_runtime qnn_dlc --truncate_64bit_io --quantize_full_type int16
                                  --quantize_io
  ```

  Tận dụng phần cứng FP16 nguyên bản của HTP, giải pháp này giữ trọn vẹn 100% độ chính xác như CPU FP32 gốc, không bị sai số tích lũy và đạt tỷ lệ offload toán tử tuyệt đối 100% lên NPU.

3. Các Cơ chế Tối ưu hóa Bộ nhớ Chuyên sâu trên Qualcomm HTP:

- *Cấp phát bộ nhớ tĩnh tuyệt đối (Static Memory Layout):* Kích thước mọi tensor được ghim cố định từ sóng âm $[1, 240240] \rightarrow$ phân khung $[1, 1500, 400] \rightarrow$ phổ DFT $[1, 1500, 201] \rightarrow$ Mel $[1, 1500, 80] \rightarrow$ Subsampling $[1, 375, 512] \rightarrow$ Encoder $[1, 375, 768] \rightarrow$ CTC Head $[1, 375, 2000] \rightarrow$ CTC Collapse $[1, 375] \rightarrow$ Byte Stream $[1, 375, 12]$. Trình biên dịch QNN phân bổ trước toàn bộ địa chỉ trên SRAM/VTCM, loại bỏ hoàn toàn chi phí xin cấp phát bộ nhớ động trong lúc chạy.

- *Truy xuất từ điển tức thời qua bộ nhớ nội tại VTCM:* Ma trận byte tĩnh $M_{\text{byte}} \in \mathbb{R}^{2000 \times 12}$ (số hóa từ từ điển SentencePiece 2.000 tokens BPE) được nhúng thẳng dưới dạng tensor hằng số. NPU dùng toán tử `Gather` tra cứu song song theo mảng Token ID. Do M_{byte} nằm trực tiếp trong bộ nhớ VTCM với băng thông cực lớn, phép ghép chuỗi byte UTF-8 diễn ra tức thời trong thời gian chưa đầy $1\,\mu$s.

- *Triệt tiêu điểm nghẽn truyền dữ liệu trên Bus hệ thống:* NPU chỉ cần chuyển duy nhất mảng số nguyên byte kết quả (≈ 4.5 KB) về CPU thay vì ma trận xác suất khổng lồ (≈ 3 MB). Kích thước dữ liệu truyền tải giảm gần $700\times$, triệt tiêu hoàn toàn hiện tượng nghẽn băng thông trên đường truyền bus PCIe/AXI của thiết bị nhúng.

3.3 Những khó khăn và Cách khắc phục

Quá trình đưa đồ thị tĩnh Zipformer lên bo mạch thực tế Qualcomm Dragonwing IQ-9075 EVK đã đối mặt và giải quyết triệt để 5 vấn đề kỹ thuật chuyên sâu:

1. Khắc phục lỗi bão hòa biên độ sóng âm (Hiện tượng audio bị "câm" hoàn toàn):

- *Nguyên nhân gốc:* Ban đầu, tín hiệu sóng âm vô tình bị nhân với hệ số 32768 theo quy ước mã hóa PCM 16-bit của Kaldi. Khi đi qua phép nhân ma trận biến đổi Fourier 512 điểm ($\text{real}^2 + \text{imag}^2$), năng lượng phổ vọt lên trên 10^9, gây tràn dải động của bộ lọc Log-Mel trên NPU và làm đầu ra mô hình bị câm hoàn toàn.

- *Giải pháp:* Loại bỏ hoàn toàn bước nhân tỉ lệ, quy chuẩn tín hiệu âm thanh đầu vào về dải biên độ thực tế tiêu chuẩn $[-1.0, 1.0]$.

2. Cơ chế Masking tĩnh triệt tiêu Token rác ở các khung đệm (Padding Frames):

- *Nguyên nhân gốc:* Các đoạn âm thanh ngắn (4–8 giây) khi đệm vào khung cố định 15 giây (240.240 mẫu) sẽ tạo ra các khung tĩnh lặng phía sau. Do thiếu mặt nạ độ dài thực tế `encoder_out_lens`, CTC Head vẫn cố gắng phân loại các khung rỗng này và sinh ra một chuỗi ký tự rác ngẫu nhiên ở cuối câu.
- *Giải pháp:* Nhóm tích hợp tensor mặt nạ độ dài thực tế kết hợp toán tử `Where` để ép toàn bộ các khung đệm vượt ngưỡng độ dài về giá trị rỗng trước khi đưa vào khối dồn token (CTC Collapse).

3. Khắc phục hiện tượng mất từ đầu câu (First-Word Dropping):

- *Nguyên nhân gốc:* Ở đầu câu thoại, người nói thường bắt đầu lấy hơi nên năng lượng âm học ban đầu còn nhỏ. Thuật toán lượng tử hóa sau huấn luyện (PTQ) chuẩn 8-bit làm tròn các giá trị biên độ nhỏ này về 0, khiến các âm tiết đầu tiên bị gán nhầm thành token rỗng (`<blank>`).
- *Giải pháp:* Nâng cấu hình lượng tử hóa kích hoạt lên W16A16 và biên dịch Native FP16 trên Hexagon HTP, khôi phục hoàn toàn độ nhạy dải động và giữ nguyên vẹn 100% các từ mở đầu.

4. Phẫu thuật đồ thị vá lỗi toán tử Boolean trên Qualcomm HTP (`0xc26`):

- *Nguyên nhân gốc:* Nhân thực thi phần cứng HTP của Qualcomm được tối ưu chuyên sâu cho các phép tính số học (INT8, INT16, FP16, INT32) và từ chối xử lý kiểu dữ liệu nhị phân `bool` trên các toán tử phân mảng như `Slice`.
- *Giải pháp:* Nhóm viết script tự động duyệt cây đồ thị ONNX (`wrap_bool_slice`), xác định các toán tử `Slice` có đầu vào là boolean mask và chèn một nút `Cast(to=INT32)` ngay phía trước, chuyển đổi mặt nạ về kiểu số nguyên 32-bit tương thích hoàn hảo với tập lệnh HTP.

5. Phân tích hiện tượng từ ngoại lai ngoài tập từ vựng (Out-of-Vocabulary – OOV):

- *Nguyên nhân gốc:* Mẫu thử nghiệm số 2 chứa tên riêng bóng bầu dục ngoại lai ("*springboks*") không nằm trong tập từ điển BPE 2.000 tokens, bị mô hình dự đoán thành "*SPRI BOOK*", làm tỷ lệ WER của riêng câu này vọt lên 25%.
- *Giải pháp:* Kiểm chứng đối chiếu cho thấy bản CPU FP32 gốc cũng gặp lỗi tương tự; nhóm xác định đây là giới hạn kích thước từ điển của checkpoint huấn luyện trước, không phải lỗi do quá trình lượng tử hóa hay phần cứng NPU.

3.4 Mã nguồn và Cách triển khai

Toàn bộ hệ thống mã nguồn của Zipformer được tổ chức khoa học theo 5 nhóm chức năng chuyên biệt:

1. Đánh giá Đối đầu Ban đầu (`src/step1_asr/`):

- `fetch_asr_data.py`: Tự động tải tập dữ liệu âm thanh chuẩn FLEURS Tiếng Việt về thư mục `data/asr/`.
- `mix_asr_noise.py`: Phối trộn tạp âm nhà xưởng ở 6 mức tỉ số tín hiệu trên nhiễu (SNR: Sạch, 20 dB, 15 dB, 10 dB, 5 dB, 0 dB) tạo tập kiểm thử `data/asr_mixed/`.
- `test_asr_zipformer.py`: Đo đạc độ chính xác WER và tốc độ RTF của Zipformer-30M thông qua bộ runtime `sherpa-onnx`.
- `test_asr_vi.py`, `test_asr_moonshine.py`, `test_asr_qwen.py`, `run_all_asr.py`: Tập hợp các kịch bản kiểm thử đối chứng toàn diện với PhoWhisper, Moonshine và Qwen3.

2. Đóng gói Đồ thị Tĩnh & Vá Lỗi Phần cứng (`step4_hardware/`):

- `build_zip150_full.py`: Kịch bản trung tâm hợp nhất toàn bộ 4 khối (Fbank DSP, Encoder 150M, CTC Head, Static CTC Collapse, UTF-8 Detokenizer) thành một đồ thị tĩnh duy nhất `zip150_full_pipeline.onnx`.
- `prepare_zipformer_for_qnn.py`: Chứa giải thuật "phẫu thuật" đồ thị tự động dò tìm và chèn node `Cast(bool → int32)` cho các toán tử `Slice` (`wrap_bool_slice`), loại bỏ lỗi crash phần cứng HTP.
- `fbank_matmul_verify.py`: Tính toán offline và kiểm chứng các ma trận hằng số DSP (cửa sổ Povey, ma trận DFT thực/ảo, bộ lọc Mel) trước khi nhúng vào đồ thị ONNX.
- `build_zip150_ctc_static.py`: Tích hợp trọng số Linear CTC Head ($W_{\text{ctc}} \in \mathbb{R}^{768\times2000}$) và ghim cố định kích thước tensor đầu vào $[1, 1500, 80]$.

3. Biên dịch Qualcomm AI Hub & Thẩm định Silicon (`src/step4_quantization/asr/zipformer/`):

- `deploy_zipformer_hub.py`: Điều phối kết nối API Qualcomm AI Hub, cấu hình cờ biên dịch QNN DLC Native FP16 tối ưu cho kiến trúc `Dragonwing IQ-9075 EVK`.
- `submit_zipformer_to_aihub.py`: Tự động hóa quy trình nộp đồ thị tĩnh hợp nhất, sinh tệp QNN Context Binary và nộp chuỗi Job (Compile `jgd6x4oep`, Profile `j56oym105`, Inference `jpv26mwjg`).
- `run_silicon_validation_fp16.py`: Kịch bản thực thi suy luận trên chip silicon thật với 15 tệp âm thanh đa điều kiện (Clean, SNR 5 dB, SNR 0 dB).

4. Module Định nghĩa Kiến trúc Lõi (`vendored_zipformer/`):

- `zipformer.py`, `subsampling.py`, `scaling.py`, `encoder_interface.py`: Các module mã nguồn độc lập định nghĩa cấu trúc đa tỷ lệ thời gian của Zipformer Encoder, tách biệt hoàn toàn khỏi các thư viện cài đặt ngoài cồng kềnh.

5. Tệp Dữ liệu Đo kiểm Thực tế (`results/`):

- `hub_silicon_test_results.json`: Nhật ký chi tiết chứa các chỉ số đo đạc phần cứng (độ trễ 184.31 ms, RAM đỉnh 7.78 MB) và từng chuỗi văn bản nhận dạng trên silicon.

- `hub_silicon_test_results.csv`: Bảng thống kê đối chiếu giữa văn bản gốc (Ground-Truth), câu dự đoán (Hypothesis) và tỷ lệ WER trên 15 mẫu vật lý.
- `eval_vi_references.json`: Nhãn văn bản tham chiếu chuẩn phục vụ đánh giá tự động.

Quy trình thực thi dòng lệnh hoàn chỉnh:

```
# Bước 1: Chuẩn bị dữ liệu và đo benchmark đối đầu tại Step 1
python src/step1_asr/fetch_asr_data.py
python src/step1_asr/mix_asr_noise.py
python src/step1_asr/test_asr_zipformer.py

# Bước 2: Đóng gói đồ thị tĩnh Single Static DAG và vá lỗi toán tử boolean
python src/step4_quantization/asr/zipformer/step4_hardware/build_zip150_full.py

# Bước 3: Nộp đồ thị hợp nhất và biên dịch Native FP16 lên Qualcomm AI Hub
python src/step4_quantization/asr/zipformer/submit_zipformer_to_aihub.py

# Bước 4: Đo kiểm Profile hiệu năng và độ chính xác WER trên chip silicon thật
python src/step4_quantization/asr/zipformer/run_silicon_validation_fp16.py
```

3.5 Kết quả từ Qualcomm AI Hub

Đồ thị tĩnh Zipformer-150M FP16 (Target Model ID: `mnz009rzm`) được đo kiểm chính thức trên bo mạch công nghiệp **Qualcomm Dragonwing IQ-9075 EVK** thông qua Qualcomm AI Hub.

Table 3: Đo đạc hiệu năng phần cứng Zipformer trên chip Dragonwing IQ-9075

Thông số kỹ thuật phần cứng	**Giá trị thực tế trên Silicon**
Mã Job Compile / Profile	`jgd6x4oep` / `j56oym105`
Thời gian suy luận NPU (Inference Latency)	**184.31 ms** (cho 15 giây âm thanh)
Hệ số thời gian thực (Real-Time Factor – RTF)	**0.0123** (nhanh hơn $81\times$ thời gian thực)
Dung lượng RAM đỉnh (Peak Memory)	**7.78 MB** (chính xác: 7.778 MB)
Thời gian nạp mô hình nguội (First Load Time)	910.99 s (tối ưu hóa bộ nhớ đệm DLC)
Thời gian nạp mô hình ấm (Warm App Load)	**536.80 ms**
Tỷ lệ tính toán trên NPU (NPU Offload)	**100.0%** (0% CPU fallback)

1. Phân tích Hiệu năng Phần cứng: Với thời gian suy luận thực tế trên silicon chỉ **184.31 ms** cho khung âm thanh 15 giây, mô hình đạt tốc độ xử lý nhanh gấp 81 lần thời gian thực (RTF $= 0.0123$). Dung lượng RAM chiếm dụng đỉnh cực thấp (**7.78 MB**) khẳng định kiến trúc Single Static DAG đã tận dụng triệt để bộ nhớ đệm cực nhanh SRAM/VTCM của Hexagon NPU v73, hoàn toàn giải phóng bộ nhớ hệ thống cho các tiến trình khác.

2. Đánh giá Định lượng Từng Mẫu Dự đoán trên Silicon: Dữ liệu trích xuất trực tiếp từ nhật ký suy luận phần cứng (`hub_silicon_test_results.json`) cho thấy các đặc trưng ngữ âm nổi bật:

Table 4: Tỷ lệ lỗi từ (WER) đo kiểm thực tế trên chip Silicon (Job: `jpv26mwjg`)

Điều kiện kiểm thử	**Số lượng mẫu**	**WER Trung bình**	**Độ chính xác tương đương**
Môi trường Sạch (Clean)	5 mẫu	7.47%	92.53%
Nhiễu công nghiệp SNR = 5 dB	5 mẫu	**6.22%**	**93.78%**
Nhiễu công nghiệp SNR = 0 dB	5 mẫu	8.69%	91.31%
Toàn bộ 15 mẫu thử nghiệm	**15 mẫu**	**7.46%**	**92.54%**

- *Độ chính xác tuyệt đối trên các câu thuần Việt dài:* Các câu hội thoại tiếng Việt thông thường (như mẫu 4 về lễ tang tại quảng trường Peter, mẫu 5 về thị thực các quốc gia châu Âu) đạt độ chính xác tuyệt đối **0.00% WER** trên cả môi trường Clean và SNR 5 dB; riêng mẫu 5 giữ vững **0.00% WER ngay cả ở mức nhiễu cực đoan SNR 0 dB**. Mẫu 1 (văn hóa cổ xưa thuần Việt) đạt WER xuất sắc 4.35%.
- *Độ bền nhiễu vượt trội trong môi trường công nghiệp:* Khi mức tạp âm tăng cao từ Sạch lên SNR 5 dB và SNR 0 dB, tỷ lệ WER toàn tập chỉ biến thiên nhẹ từ 7.47% lên 8.69%. Tại mức 5 dB, WER thậm chí đạt mức tốt nhất 6.22% nhờ cơ chế lọc nhiễu dải động của ma trận bộ lọc Mel trong khối Fbank DSP.
- *Phân tích nguồn sai số ngoài từ điển (Out-of-Vocabulary – OOV):* Điểm số WER trung bình bị kéo tăng chủ yếu do mẫu số 2 chứa tên riêng bóng bầu dục ngoại lai ("*springboks*" bị dự đoán thành "*SPRI BOOK*" ở clean và "*SRIK*" ở 5 dB, đẩy WER cục bộ câu này lên 18.75%–25.0%) và mẫu số 3 chứa từ mượn ("*t rex*" thành "*TBE*"). Đây là hạn chế vốn có về kích thước tập từ vựng BPE ($V = 2000$) của checkpoint gốc (bản CPU FP32 cũng gặp lỗi tương tự), không phải sai số do quá trình biên dịch hay thực thi trên NPU Qualcomm.

Kết quả thực nghiệm trên bo mạch vật lý xác nhận giải pháp đồ thị tĩnh Native FP16 duy trì trọn vẹn chất lượng nhận diện của mô hình gốc, bảo đảm tính sẵn sàng cao độ cho các thiết bị dịch thuật công nghiệp.

4 Model SenseVoice (En/Zh/Ko)

4.1 Kiến trúc Model

Để phục vụ bài toán nhận dạng giọng nói đa ngữ thời gian thực cho 3 ngoại ngữ: **Tiếng Anh (EN), Tiếng Trung (ZH) và Tiếng Hàn (KO)**, dự án lựa chọn mô hình **SenseVoice-Small** (Alibaba FunASR). Khác với các kiến trúc tự hồi quy truyền thống phải dự đoán từng từ tuần tự, SenseVoice vận hành theo cơ chế **phi tự hồi quy (Non-autoregressive – NAR)**, giải mã toàn bộ câu thoại trong một lượt truyền thẳng duy nhất (*single-pass forward*).

Để triệt tiêu hoàn toàn sự phụ thuộc vào CPU Host, hệ thống được thiết kế theo kiến trúc **Split-Graph (2 đồ thị NPU tuần tự)** kết nối liên hoàn 5 khối chức năng chạy 100% trên phần cứng NPU Qualcomm Hexagon:

$$\text{Waveform (29s)} \xrightarrow[\text{FP16}]{\text{Graph 1: Frontend}} \text{Fbank} \xrightarrow[\text{W8A16}]{\text{Graph 2: Encoder + CTC + Detok}} \text{Luồng Byte UTF-8}$$

1. Khối WavFrontend DSP Tĩnh (Graph 1 – 82 toán tử trên NPU): Đảm nhận nhiệm vụ chuyển đổi sóng âm thô 16 kHz (khung đệm tối đa 29 giây, tương đương 464.000 mẫu) thành ma trận đặc trưng phổ âm thanh $F_{\text{bank}} \in \mathbb{R}^{1\times500\times560}$:

- *Phân khung & Lấy mẫu:* Dùng lớp `Conv1D` (cửa sổ 400 mẫu, bước nhảy 160 mẫu) trích xuất đồng thời 500 khung thoại song song.
- *Chuẩn hóa biên độ động thích ứng theo khung (Per-frame Normalization):* Tính toán hệ số co giãn động $g_t = 2/\max(\text{peak}_t, 10^{-4})$ cho từng khung nhằm ép tín hiệu nằm gọn trong dải an toàn của phần cứng, triệt tiêu nguy cơ tràn số.
- *Biến đổi Fourier ma trận (DFT MatMul):* Thay thế hàm biến đổi Fourier động (`torch.fft.rfft`) bằng phép nhân ma trận với hai hằng số trực giao Thực/Ảo ($W_{\text{real}}, W_{\text{imag}} \in \mathbb{R}^{512\times257}$), tính toán phổ năng lượng hoàn toàn qua các lõi Tensor Cores.
- *Bộ lọc Mel & Bù trừ Logarithm:* Chiếu qua ma trận Mel 80 kênh và lấy log tự nhiên kèm phép bù trừ năng lượng giải tích: $\ln(\text{mel}) - 2\ln(g_t) + 2\ln(32768)$.
- *Ghép khung LFR & Chuẩn hóa CMVN:* Áp dụng kỹ thuật xếp chồng khung thời gian (Linear Frequency Residual – Stack 7, Hop 6) và chuẩn hóa trung bình phương sai tĩnh (CMVN), xuất ra tensor đặc trưng $[1, 500, 560]$.

2. Khối SenseVoice Transformer Core (50 lớp sâu): Đóng vai trò là khối xử lý trung tâm giải mã ngữ âm:

- *Tĩnh hóa mã hóa vị trí (Static Positional Encoding):* Ma trận biểu diễn vị trí $[1, 504, 560]$ được cố định hóa thành `Constant Tensor` trong đồ thị ONNX nhằm ngăn chặn lỗi suy diễn kích thước động trên trình điều khiển QAIRT.
- *50 lớp SAN-M Transformer:* Mạng nơ-ron sâu gồm 50 tầng tự chú ý có bộ nhớ (Self-Attention Network with Memory – SAN-M). Khối tiếp nhận đồng thời ma trận Fbank cùng các token điều khiển hệ thống gồm mã nhận diện ngôn ngữ (Language ID: Trung $= 3$, Anh $= 4$, Hàn $= 12$) và chế độ chuẩn hóa văn bản ngược (Text Normalization – ITN), chiếu ngữ cảnh toàn câu về không gian ẩn $[1, 504, 512]$.

3. Khối CTC Projection Head & ArgMax: Chiếu biểu diễn ẩn sang không gian xác suất của tập từ vựng đa ngữ kích thước $V = 25.055$ tokens qua một lớp Linear Projection thuần túy ($[1, 504, 512] \times [512, 25055] \to [1, 504, 25055]$). Phép toán arg max được Vector Processing Unit (VPU) thực thi song song theo trục từ vựng để rút ra mảng token dự đoán ban đầu $[1, 504]$.

4. Khối Static CTC Collapse với Cơ chế Thùng rác Tĩnh (Trash-Bin Index 504): Khắc phục triệt để việc thiếu cấu trúc vòng lặp điều kiện động trên NPU bằng giải thuật ma trận hóa:

- *Mặt nạ độ dài thực tế (Length Masking):* Dùng tensor độ dài sóng âm đầu vào (`wav_len`) để ép toàn bộ các token nằm ngoài phạm vi nói thực tế về giá trị 0, loại bỏ hiện tượng sinh từ rác ở khoảng lặng cuối câu.
- *Cơ chế Trash-Bin Scatter:* Thiết lập mặt nạ nhị phân trên tập từ điển 25.055 classes để nhận diện các token lặp liền kề, token rỗng (`<blank>`), và 171 thẻ điều khiển hệ

thống đặc biệt (như <|lid|>, <|nospeech|>, <|itn|>). Thay vì xóa động, NPU phân luồng toàn bộ các token không hợp lệ này về một vị trí cố định gọi là **Thùng rác tĩnh (Trash-Bin Index 504)**, đồng thời dùng toán tử `CumSum` dồn các từ vựng ngữ nghĩa hợp lệ về đầu mảng cố định $[1, 504]$.

5. Khối Zero-CPU Static Byte Detokenizer (Giải mã Byte Đa ngữ trên NPU):

- *Nhúng bảng mã tĩnh vào bộ nhớ nội tại VTCM:* Toàn bộ từ điển 25.055 tokens được số hóa thành ma trận byte UTF-8 hằng số $M_{\text{byte}} \in \mathbb{R}^{25055 \times 24}$ và nạp trực tiếp vào bộ nhớ siêu tốc VTCM của NPU Hexagon.
- *Tính tương thích đa ngữ hoàn hảo của chuẩn UTF-8:*
  - **Tiếng Anh (ASCII/Latin):** Mã hóa chuẩn bằng 1 byte duy nhất cho mỗi ký tự.
  - **Tiếng Trung (Hán tự) & Tiếng Hàn (Hangul):** Chuẩn quốc tế UTF-8 mã hóa mỗi chữ Hán và mỗi âm tiết Hangul bằng đúng **3 bytes** (ví dụ: chữ Hán "*nhĩ*" $\rightarrow [232, 191, 153]$; âm tiết Hangul "*an*" $\rightarrow [235, 139, 164]$).

  Độ dài đệm $L_{\text{max}} = 24$ bytes (căn lề bội số 4 byte tối ưu cho tập lệnh mở rộng Hexagon Vector Extensions – HVX) giúp một bảng tra cứu tĩnh duy nhất bao phủ trọn vẹn cả 3 ngôn ngữ mà không cần cấu trúc rẽ nhánh phức tạp.
- *Giải mã tức thì tại Host (< 0.001 ms):* NPU sử dụng toán tử `Gather` tra cứu trực tiếp và xuất ra luồng byte $[1, 12096]$ (504×24). Host CPU chỉ việc gọi hàm nguyên bản của hệ thống `bytes.decode('utf-8')` để hiển thị văn bản, loại bỏ 100% các thư viện tokenizer nặng nề khỏi thiết bị biên.

4.2 Quantize và Deploy

Để đưa mô hình Transformer 50 lớp quy mô lớn lên bộ vi xử lý Qualcomm Hexagon NPU v73 HTP, dự án đã trải qua quá trình nghiên cứu và tối ưu hóa chuyên sâu:

1. Thất bại của Mô hình Gộp Đơn Đồ thị (Single Unified Graph): Ban đầu, nhóm thử nghiệm đóng gói toàn bộ 5 khối vào một đồ thị tĩnh duy nhất và thực hiện lượng tử hóa sau huấn luyện (PTQ W8A16). Mặc dù mô hình biên dịch thành công và đạt tỷ lệ offload 100% trên NPU, nhưng kết quả đo kiểm trên chip silicon thật lại bị suy sụp hoàn toàn (độ chính xác 0%: Tiếng Trung và Tiếng Hàn bị nén cụt thành dấu chấm câu, Tiếng Anh bị mất từ nghiêm trọng).

2. Chẩn đoán Giải phẫu Đồ thị (Graph Surgery & Bypass Analysis): Nhóm tiến hành phân tích độ tương đồng cosin (Cosine Similarity) qua 142 tầng chuẩn hóa LayerNorm và thực hiện thí nghiệm cách ly (*Frontend Bypass Test*): Thay thế tầng đầu ra của Frontend bằng giá trị FP32 tính từ CPU trong khi giữ nguyên 50 tầng Transformer lượng tử hóa W8A16. Kết quả ghi nhận Character Error Rate (CER) của Tiếng Trung lập tức phục hồi từ 100% xuống còn **8.1%**. Phát hiện này chứng minh: **Lỗi suy thoái không nằm ở 50 tầng Transformer, mà bắt nguồn từ việc dải động của phổ âm thanh DSP bị biến dạng khi ép qua lượng tử hóa tĩnh.**

3. Đột phá Kiến trúc Split-Graph (2 Đồ thị Tuần tự): Từ kết luận chẩn đoán, nhóm tái cấu trúc pipeline thành 2 đồ thị NPU chuyên biệt:

- **Graph 1 – Frontend v3 (Native FP16 trên NPU):** Dải năng lượng phổ Mel trải rộng tới 11 bậc độ lớn ($10^{-9} \dots 95$), không thể chịu đựng sai số làm tròn của thang đo lượng tử hóa 8-bit. Do đó, Frontend được giữ nguyên ở định dạng **Native FP16** (82 toán tử), biên dịch trực tiếp bằng lệnh:

  ```
  --target_runtime qnn_dlc --truncate_64bit_io
  ```

  Kết hợp giải thuật chuẩn hóa per-frame, sai số mô phỏng FP16 giảm từ 1.75 xuống chỉ còn 0.0034, đạt độ tương đồng cosin trên chip silicon thực tế lên tới **0.99999** so với bản CPU FP32 gốc.

- **Graph 2 – Transformer Core & Classifier (W8A16 Mixed Precision):** Toàn bộ 50 tầng Transformer và khối Detokenizer (3.409 toán tử) được lượng tử hóa theo cơ chế **W8A16 Mixed Precision**:

  - *Trọng số mạng (Weights INT8):* Nén dung lượng lưu trữ từ 893 MB (PyTorch FP32) xuống chỉ còn **233 MB**, giảm tải 74% băng thông đọc ghi từ bộ nhớ.
  - *Kích hoạt nơ-ron (Activations INT16):* Duy trì dải động 65.536 mức rời rạc cho toàn bộ các phép tính Attention và FFN, triệt tiêu hiện tượng bão hòa dải động của chuẩn INT8 truyền thống.
  - Đồ thị được hiệu chuẩn (calibrate) bằng 25 mẫu trong miền Fbank chuẩn sinh ra từ dữ liệu thực tế và biên dịch với cờ tối ưu:

    ```
    --target_runtime qnn_dlc --quantize_io --truncate_64bit_io
    ```

4. Giao tiếp Trung gian Zero-overhead giữa 2 Đồ thị: Hai đồ thị giao tiếp thông qua bộ đệm RAM chia sẻ của Host: Graph 1 xuất tensor Fbank $[1, 500, 560]$, tầng điều phối chuyển giao trực tiếp con trỏ bộ nhớ làm đầu vào cho Graph 2 mà không thực hiện bất kỳ phép toán trung gian nào trên CPU. Thời gian chuyển tiếp đo được < 0.001 ms, bảo đảm toàn bộ chuỗi suy luận vận hành liền mạch với hiệu năng tương đương một đồ thị đơn nhất.

4.3 Những khó khăn và Cách khắc phục

Quá trình đưa SenseVoice-Small lên bo mạch Qualcomm Dragonwing IQ-9075 EVK đã đối mặt và giải quyết triệt để 4 vấn đề kỹ thuật then chốt:

1. Thất bại ban đầu của mô hình gộp 5 khối (Độ chính xác trên silicon gần 0%):

- *Nguyên nhân gốc:* Đóng gói toàn bộ 5 khối chức năng vào một đồ thị ONNX đơn nhất và lượng tử hóa W8A16. Dải động của khối tiền xử lý âm thanh DSP trải rộng tới 11 bậc độ lớn ($10^{-9} \dots 95$), không thể dung nạp sai số làm tròn của thang đo lượng tử hóa 1-scale, khiến phổ Mel bị biến dạng hoàn toàn và làm sụp đổ mô hình (tiếng Trung/Hàn bị câm chỉ ra dấu chấm, tiếng Anh bị ngắt cụt câu).
- *Giải pháp:* Thực hiện thí nghiệm cách ly (*Frontend Bypass Test*) xác định 50 tầng Transformer hoàn toàn bình thường; từ đó nhóm chuyển dịch sang kiến trúc **Split-Graph**

(2 đồ thị NPU tuần tự): Graph 1 (Frontend) chạy Native FP16 không lượng tử hóa; Graph 2 (Encoder) chạy W8A16 Mixed Precision.

2. Lọt 171 thẻ điều khiển hệ thống và tràn bộ đệm ký tự đa byte UTF-8:

- *Nguyên nhân gốc:* Các token đặc biệt của mô hình (như <|lid|>, <|itn|>) chưa được che chắn nên xuất hiện lẫn trong chuỗi kết quả; đồng thời bộ đệm ký tự ban đầu quá nhỏ ($L_{\text{max}} = 12 \sim 16$) khiến tiếng Trung và Hàn (mỗi chữ tốn 3 bytes UTF-8) bị tràn bộ đệm và cắt cụt câu.
- *Giải pháp:* Quét toàn bộ từ điển 25.055 classes, tạo mặt nạ nhị phân tống toàn bộ 171 token điều khiển vào **Thùng rác tĩnh (Trash-Bin Index 504)**; đồng thời nâng độ dài đệm lên $L_{\text{max}} = 24$ bytes (căn lề 4 byte cho tập lệnh HVX) với kích thước chuỗi byte cố định $504 \times 24 = 12.096$ bytes.

3. Hiện tượng Blank Dominance và lỗi crash toán tử Boolean trên HTP:

- *Nguyên nhân gốc:* Ép âm thanh ngắn 4–8 giây vào khung đệm cố định 29 giây khiến 70–85% dữ liệu là số 0, làm lệch thang đo Min-Max của bộ hiệu chuẩn PTQ và khiến token rỗng (<blank>) có bias áp đảo nuốt chửng các âm vị; ngoài ra mặt nạ lọc để kiểu boolean khiến phần cứng HTP báo lỗi `0xc26` do từ chối toán tử `Gather BOOL_8`.
- *Giải pháp:* Thiết kế cơ chế *Multi-bucket calibration* chia dữ liệu theo 6 bậc độ dài thực tế để thu thập đúng phân phối năng lượng giọng nói; đồng thời chuyển đổi toàn bộ mặt nạ lọc sang kiểu số nguyên `int32` nguyên bản trên NPU.

4. Lỗi tràn số trên (Overflow) và tràn số dưới (Underflow) trên phần cứng FP16:

- *Nguyên nhân gốc:* Vi xử lý Hexagon HTP tính toán số thực thuần túy trên miền FP16 (Max $= 65.504$, Min $= 6.1 \times 10^{-5}$). Nếu nhân tín hiệu với hệ số Kaldi 32768, năng lượng phổ tại tầng FFT vượt qua 10^9, gây tràn số trên thanh ghi NPU; nếu bỏ nhân thì tín hiệu quá nhỏ gây tràn số dưới khiến 51.8% dữ liệu Mel rơi xuống dưới ngưỡng Min và bị ép về số 0 tuyệt đối.
- *Giải pháp (Đột phá Frontend v3):* Thiết kế giải thuật **Chuẩn hóa biên độ động theo từng khung thoại** ($g_t = 2/\max(\text{peak}_t, 10^{-4})$), ép toàn bộ phép tính FFT và Mel filterbank nằm an toàn trong dải $[10^{-3}, 5000]$ của FP16, sau đó bù trừ chính xác bằng giải tích sau hàm log. Tensor Fbank trên silicon NPU thật đạt độ tương đồng tuyệt đối **0.99999** so với CPU gốc.

4.4 Mã nguồn và Cách triển khai

Toàn bộ hệ thống mã nguồn của SenseVoice được phân định khoa học theo 6 nhóm chức năng phục vụ trọn vẹn chu trình phát triển:

1. Đánh giá Đối đầu Ban đầu (`src/step1_asr/`):

- `fetch_asr_data.py`: Tự động tải tập dữ liệu chuẩn FLEURS cho 3 ngoại ngữ (Anh, Trung, Hàn) về thư mục `data/asr/`.
- `mix_asr_noise.py`: Phối trộn tạp âm công nghiệp ở các mức SNR từ 0 dB đến 20 dB.

- `test_asr_multi.py`: Thực nghiệm đo lường WER/CER và RTF của SenseVoice-Small trên môi trường CPU/GPU.
- `test_asr_moonshine.py`, `test_asr_qwen.py`, `run_all_asr.py`: Kịch bản kiểm thử đối chứng với các mô hình ứng viên bị loại bỏ (Moonshine, Qwen3).

2. Đóng gói & Xuất Đồ thị ONNX (`src/step4_quantization/asr/sensevoice/export/`):

- `sv_export_frontend_v3_perframe.py`: Xuất Graph 1 (Frontend v3 tích hợp thuật toán chuẩn hóa biên độ động Per-Frame) sang ONNX FP16.
- `step4_s1_export_sensevoice_split_npu.py`: Xuất Graph 2 (Encoder 50 tầng + CTC Classifier + Thùng rác tĩnh Trash-Bin Index 504 + Zero-CPU Detokenizer).
- `step4_s1_export_sensevoice_e2e_unified.py`: Bản thiết kế đồ thị đơn 5 khối nguyên khối (phiên bản thử nghiệm ban đầu v1/v2).
- `inline_onnx.py`: Công cụ nén các tensor hằng số và tối ưu hóa cấu trúc đồ thị ONNX.

3. Lượng tử hóa & Biên dịch Qualcomm AI Hub (`src/step4_quantization/asr/sensevoice/hub/`):

- `sv_prepare_calib_fbank.py`: Tạo tập dữ liệu hiệu chuẩn 25 mẫu trong miền Fbank phục vụ lượng tử hóa W8A16 cho Encoder.
- `submit_full_sensevoice_pipeline.py`: Tự động hóa toàn bộ chuỗi Job trên AI Hub: Biên dịch Graph 1 (FP16 DLC), lượng tử hóa Graph 2 (W8A16) và biên dịch Context Binary cho Hexagon v73 HTP.
- `sv_chain_v3_frontend.py`: Kịch bản nộp suy luận chuỗi liên hoàn Graph 1 $\rightarrow$ Graph 2 trực tiếp trên cloud device của Qualcomm.
- `sv_prepare_calib_v2.py`: Công cụ sinh dữ liệu hiệu chuẩn đa bậc (Multi-bucket calibration).

4. Thẩm định Thực nghiệm trên Chip Silicon (`eval/`):

- `eval/fetch_asr_eval30.py`: Tải 90 mẫu âm thanh độc lập (30 EN, 30 ZH, 30 KO) từ tập `test[5:]` của Google FLEURS.
- `eval/sv_eval30_cpu_ref.py`: Chạy suy luận đối chiếu chuẩn FP32 trên ONNX Runtime CPU.
- `eval/sv_eval30_npu.py`: Chạy suy luận trực tiếp trên chip silicon NPU thông qua Qualcomm AI Hub API.
- `eval/sv_eval30_common.py`: Thư viện chuẩn hóa chuỗi và tính toán CER/WER đa ngữ độc lập.

5. Chẩn đoán & Giải phẫu Đồ thị Phần cứng (`diagnostics/`):

- `diagnostics/sv_a1_cosine_compare.py`: Đo đạc Cosine Similarity qua 142 tầng LayerNorm để phát hiện điểm phân kỳ số học giữa CPU và NPU.

- `diagnostics/sv_a3_bypass_frontend.py`: Script thực thi Bypass Test A3 cô lập nguồn lỗi giữa tầng DSP và tầng Transformer.
- `diagnostics/sv_sim_perframe_norm.py`, `diagnostics/sv_scan_scale_range.py`: Mô phỏng giải thuật chuẩn hóa per-frame và quét dải động số học trước khi nạp lên phần cứng.

6. Tệp Dữ liệu Kết quả Đo kiểm Thực tế (`results/`):

- `hub_silicon_test_results.json`: Nhật ký đo đạc chi tiết trên chip silicon (độ trễ 269.40 ms, RTF 0.0093, tỷ lệ lỗi 15 mẫu thử nghiệm).
- `hub_silicon_test_summary.csv`: Bảng thống kê đối chiếu Ground-Truth và câu nhận dạng thực tế trên chip NPU.
- `eval30_npu_results.json`, `eval30_cpu_ref_results.json`: Kết quả đo kiểm toàn diện trên bộ 90 câu Google FLEURS độc lập.

Quy trình thực thi dòng lệnh hoàn chỉnh:

```
# Bước 1: Benchmark đối đầu sơ bộ tại Step 1
python src/step1_asr/fetch_asr_data.py
python src/step1_asr/test_asr_multi.py

# Bước 2: Xuất đồ thị tĩnh ONNX (Split-Graph)
python src/step4_quantization/asr/sensevoice/export/sv_export_frontend_v3_perframe.py
python src/step4_quantization/asr/sensevoice/export/step4_s1_export_sensevoice_split_npu.py

# Bước 3: Tạo tập hiệu chuẩn và nộp chuỗi biên dịch lên Qualcomm AI Hub
python src/step4_quantization/asr/sensevoice/hub/sv_prepare_calib_fbank.py
python src/step4_quantization/asr/sensevoice/hub/submit_full_sensevoice_pipeline.py

# Bước 4: Tải 90 mẫu test độc lập và đối chiếu kết quả giữa CPU FP32 và NPU Thật
python src/step4_quantization/asr/sensevoice/eval/fetch_asr_eval30.py
python src/step4_quantization/asr/sensevoice/eval/sv_eval30_cpu_ref.py
python src/step4_quantization/asr/sensevoice/eval/sv_eval30_npu.py
```

4.5 Kết quả từ Qualcomm AI Hub

Mô hình SenseVoice-Small đa ngữ theo kiến trúc Split-Graph được đo kiểm thực nghiệm toàn diện trên bo mạch công nghiệp **Qualcomm Dragonwing IQ-9075 EVK** (Hexagon NPU v73 HTP) thông qua hệ thống Qualcomm AI Hub.

1. Phân tích Hiệu năng Phần cứng & Độ trễ Thực tế: Dữ liệu trích xuất từ nhật ký đo đạc phần cứng (`hub_silicon_test_results.json`) ghi nhận:

- *Tỷ lệ thực thi phần cứng tuyệt đối (NPU Offload):* Đạt **100.00% (0% CPU fallback)**, toàn bộ 3.491 toán tử từ khâu trích xuất phổ DSP đến giải mã UTF-8 đều chạy trọn vẹn trên các lõi HTP/VPU của Hexagon NPU.

Table 5: Đo đạc hiệu năng phần cứng SenseVoice Split-Graph trên Dragonwing IQ-9075

Thành phần đồ thị	**Mã Job (Compile / Profile)**	**Toán tử**	**Phần cứng**	**Độ trễ**
Graph 1: Frontend v3 (FP16)	`jg9olx88g` / `jgd6xz0zp`	82 ops	100% NPU	**65.39 ms**
Graph 2: Encoder (W8A16)	`jp3ojw6np` / `jp0ozvw2p`	3.409 ops	100% NPU	**204.01 ms**
Zero-CPU Detokenizer	Tích hợp sẵn trong Graph 2	–	Host CPU	< 0.001 ms
Toàn bộ Pipeline	**Tổng chuỗi liên hoàn**	**3.491 ops**	**100% NPU**	**269.40 ms**

- *Độ trễ toàn trình siêu tốc:* Tổng thời gian suy luận cho khung âm thanh tối đa 29 giây chỉ tốn **269.40 ms** (Graph 1 tốn 65.39 ms và Graph 2 tốn 204.01 ms). Hệ số thời gian thực đạt mức ấn tượng:

  $$\textbf{RTF} = \mathbf{0.00929}$$ (xử lý nhanh hơn gấp **107 lần** thời gian thực).

  Với một câu thoại thông thường dài 3–5 giây, thời gian suy luận thực tế trên chip chỉ tốn khoảng **28–45 ms**, giải phóng hoàn toàn ngân sách thời gian cho các mô-đun dịch thuật và tổng hợp giọng nói.

Table 6: Độ chính xác đo kiểm thực tế trên chip Silicon NPU và đối sánh với CPU

Ngôn ngữ	**Chỉ số**	**NPU (Tập mẫu)**	**NPU (90 câu FLEURS)**	**CPU FP32 Gốc**
Tiếng Anh (EN)	WER	**7.44%**	8.98%	8.45%
Tiếng Trung (ZH)	CER	11.03%	**8.33%**	8.65%
Tiếng Hàn (KO)	CER	–	**7.33%**	7.65%
Tiếng Hàn (KO)	WER	32.28%	30.68%	26.56%

2. Đánh giá Định lượng Ngữ âm Đa ngữ trên Silicon: Đối chiếu chi tiết giữa kết quả chạy NPU thật và CPU tham chiếu (`eval30_npu_results.json`):

- *Tiếng Anh (EN) – Tỷ lệ lỗi WER 7.44%:* Trên tập kiểm thử vật lý, mô hình nhận diện chính xác các câu thoại dài và phức tạp. Đặc biệt, mẫu `en_4.wav` (câu về khủng long và chim) đạt độ chính xác tuyệt đối **0.00% WER**; mẫu `en_0.wav` đạt WER ấn tượng 5.26%. Trên bộ mở rộng 30 câu FLEURS độc lập, tỷ lệ lỗi ký tự (CER) chỉ ở mức 4.84%.

- *Tiếng Trung (ZH) – CER 8.33% (Vượt trội hơn CPU FP32):* Nhờ dải động ổn định của Frontend v3 kết hợp với W8A16 Mixed Precision, NPU tái tạo chữ Hán chuẩn xác tới từng nét. Trên 30 câu kiểm thử độc lập, CER của NPU đạt **8.33%**, tốt hơn cả bản tham chiếu FP32 CPU gốc (8.65%, giảm 0.32% lỗi). Các thực thể số học và ngày tháng được chuẩn hóa hoàn hảo nhờ khối ITN tích hợp (ví dụ: nhận diện và chuyển đổi chuẩn xác các đại lượng đo lường và mốc thời gian như "15 mét", "tháng 8 năm 2011").

- *Tiếng Hàn (KO) – CER 7.33% (Khẳng định chất lượng nhận diện âm vị):* Tiếng Hàn là ngôn ngữ chắp dính (agglutinative), trong đó trợ từ và vĩ tố dính liền vào danh từ; chỉ cần mô hình ngắt khoảng trắng khác đi một chút (ví dụ: viết số kèm trợ từ so với viết chữ đầy đủ) thì chỉ số WER sẽ bị phạt nặng (30.68%). Tuy nhiên, xét trên chỉ số lỗi ký tự **CER**, NPU đạt mốc xuất sắc **7.33%** (vượt trội hơn mức 7.65% của FP32 CPU), chứng minh toàn bộ các âm tiết Hangul cốt lõi được mô hình NPU giải mã chính xác tuyệt đối.

Kết quả đo kiểm silicon chứng minh giải pháp Split-Graph kết hợp lượng tử hóa W8A16 không những bảo toàn trọn vẹn chất lượng của mô hình gốc 893 MB mà còn nâng cao độ chính xác nhận diện trên phần cứng nhúng thực tế.

5 Demo ASR Dashboard

5.1 Mã nguồn và Cách triển khai

Để phục vụ kiểm thử tương tác trực quan và trình diễn tại hiện trường, dự án phát triển máy chủ điều khiển tập trung (**ASR Demo Dashboard**) tích hợp đồng thời cả hai mô hình NPU (Zipformer Tiếng Việt và SenseVoice Ngoại ngữ) vào một giao diện Web duy nhất.

1. Cấu trúc Hệ thống (`src/step4_quantization/asr/asr_demo_server.py`):

- *Bộ máy thực thi kép (Dual Engines):* Xây dựng trên nền tảng `FastAPI` không đồng bộ, cho phép chuyển đổi linh hoạt giữa hai động cơ: **Local CPU** (chạy các tệp ONNX nội bộ với độ trễ siêu tốc ~ 25–40 ms, hoàn toàn offline) và **NPU Dragonwing** (nộp suy luận trực tiếp lên chip Hexagon HTP qua Qualcomm AI Hub: Tiếng Việt dùng Zipformer-150M FP16 `mnz009rzm`, 184.31 ms; Ngoại ngữ dùng SenseVoice-Small Split-Graph `mn4ooxjzq` $\rightarrow$ `mq800zwjn`, 269.40 ms).
- *Giao diện Web tương tác:* Tích hợp bộ chuyển đổi động cơ thực thi, hỗ trợ thu âm trực tiếp từ microphone trình duyệt, tải tệp âm thanh (WAV/MP3), chọn nhanh các mẫu kiểm thử chuẩn (Clean, SNR 5 dB, SNR 0 dB), và giám sát phần cứng thời gian thực (độ trễ, RAM đỉnh, RTF).
- *Giải mã Zero-CPU:* Khi chạy trên NPU, máy chủ tiếp nhận mảng byte UTF-8 trực tiếp từ chip và chuyển đổi thành văn bản tức thời qua `bytes.decode('utf-8')` (< 0.001 ms) mà không tốn tải CPU.

2. Hướng dẫn Khởi chạy ASR Dashboard:

a) Cách 1 – Chạy Local (Nội bộ trên máy cá nhân): Sử dụng khi tự kiểm thử trực tiếp trên máy tính phát triển:

```
python src/step4_quantization/asr/asr_demo_server.py
```

Mở trình duyệt bất kỳ (Chrome, Edge, Firefox) và truy cập tại địa chỉ: `http://localhost:8420`. Có thể tùy biến cổng hoặc host nếu cần:

```
python src/step4_quantization/asr/asr_demo_server.py --port 9000 --host 127.0.0.1
```

b) Cách 2 – Chạy Public ra ngoài Internet bằng Cloudflare Tunnel (Có HTTPS):

Chính sách bảo mật của các trình duyệt web (Chrome, Edge, Safari trên cả iOS và Android) **chỉ cho phép ứng dụng truy cập Microphone khi kết nối thông qua `localhost` hoặc giao thức an toàn `HTTPS` (kết nối qua địa chỉ IP mạng nội bộ thường `http://192.168.x.x:8420` sẽ bị chặn quyền micro ngay lập tức). Giải pháp tối ưu là sử dụng **Cloudflare Tunnel** – hoàn toàn miễn phí, tự động cấp chứng chỉ SSL/HTTPS hợp lệ và không cần mở cổng modem (NAT/Port Forwarding).

Tùy chọn A: Tự động hoàn toàn (Khuyên dùng) – Chỉ cần thêm cờ `--share` vào lệnh khởi động:

```
python src/step4_quantization/asr/asr_demo_server.py --share
```

Script tự động phát hiện tệp thực thi `cloudflared` (hoặc tự động tải về nếu chưa có), khởi tạo đường hầm bảo mật và in ra màn hình đường dẫn công khai an toàn:

```
https://example-subdomain.trycloudflare.com
```

Người dùng chỉ cần mở đường link này trên điện thoại hoặc máy tính từ xa để thử nghiệm tính năng thu âm qua micro tức thời.

Tùy chọn B: Khởi chạy thủ công qua 2 Terminal riêng biệt:

- **Terminal 1:** Khởi động máy chủ ASR:

  ```
  python src/step4_quantization/asr/asr_demo_server.py
  ```

- **Terminal 2:** Khởi tạo đường hầm Cloudflare (yêu cầu cài đặt sẵn `cloudflared`):

  ```
  cloudflared tunnel --url http://127.0.0.1:8420
  ```

Sao chép URL có đuôi `.trycloudflare.com` xuất hiện trên terminal để truy cập từ xa.

5.2 Nhận xét kết quả

Thông qua quá trình vận hành trực tiếp Dashboard trên bo mạch **Qualcomm Dragonwing IQ-9075 EVK**, nhóm nghiên cứu ghi nhận 4 kết quả then chốt:

- **Độ trễ Siêu tốc Thoát khỏi Giới hạn Thời gian Thực:** Cả hai mô hình đều có hệ số RTF < 0.013 (Zipformer 184.31 ms/15s, SenseVoice 269.40 ms/29s), chỉ chiếm chưa đầy 10% ngân sách trễ toàn trình (≤ 3.0 giây) của hệ thống dịch giọng nói hai chiều (S2S).
- **Triệt tiêu Nghẽn Cổ chai CPU (Zero-CPU):** CPU duy trì mức tải $< 1\%$ trong suốt phiên suy luận liên tục nhờ việc đẩy 100% tính toán DSP và giải mã byte vào phần cứng NPU.
- **Kháng Nhiễu Công nghiệp Vượt Trội:** Hệ thống giữ vững độ chính xác cao ở môi trường cực ồn SNR 5 dB và 0 dB (WER Tiếng Việt 6.22%, CER Tiếng Trung 8.33%, CER Tiếng Hàn 7.33%), khối ITN tự động chuẩn hóa chính xác số từ và đơn vị đo lường.
- **Sẵn sàng Tích hợp Sản phẩm Biên (Edge Gateway):** Máy chủ vận hành ổn định, sẵn sàng đóng gói thành microservice phục vụ truyền phát dữ liệu thời gian thực cho tai nghe thông minh và thiết bị phiên dịch hiện trường.

6 Pipeline Integration

6.1 Các Modules còn lại

Bên cạnh khối ASR đã hoàn thiện 100% trên NPU, hệ thống dịch giọng nói toàn trình (S2S) tích hợp đồng bộ 3 phân hệ phụ trợ:

- **Audio Front-end (Step 0):** Mô hình DeepFilterNet3 kết hợp VAD giúp lọc sạch tạp âm công nghiệp (70–95 dB SPL) và trích xuất các phân đoạn giọng nói sạch trước khi chuyển vào ASR.
- **Dịch máy Đa ngữ – MT (Step 2):** Mô hình NLLB-200-distilled-600M nén INT8 (~ 594 MB) hỗ trợ dịch thuật hai chiều 6 hướng giữa Tiếng Việt và 3 ngoại ngữ (VI $\rightleftharpoons$ EN / ZH / KO) với độ trễ < 0.7 giây/câu.
- **Tổng hợp Giọng nói – TTS (Step 3):** Piper TTS cho Tiếng Việt (ONNX, RTF siêu tốc 0.144), Supertonic 3 cho Tiếng Anh và Tiếng Hàn, cùng MeloTTS cho Tiếng Trung.

Toàn bộ hệ thống sau khi nén chỉ chiếm ≈ 1.38 GB bộ nhớ lưu trữ, nằm gọn trong dung lượng RAM của bo mạch Qualcomm Dragonwing IQ-9075 EVK.

6.2 Hướng hợp nhất Pipeline

Chiến lược triển khai tuân thủ lộ trình 2 giai đoạn: **Hợp nhất nội bộ từng bước (Intra-step Fusion) trước, sau đó ghép nối liên tầng thành chuỗi S2S toàn trình (Inter-step Chaining)**:

1. Pha 1 – Hợp nhất Nội bộ Từng Bước (Intra-step Fusion):

- *Khối ASR:* Hợp nhất toàn bộ Fbank DSP, Encoder, CTC Head và bộ Detokenizer UTF-8 thành đồ thị tĩnh duy nhất chạy 100% NPU.
- *Khối MT:* Tĩnh hóa kích thước ngữ cảnh và nhúng bảng tra cứu Language ID vào đồ thị QNN DLC của NLLB-200.
- *Khối TTS:* Đóng gói liên hoàn Text Encoder, bộ tạo phổ Mel và Vocoder thành một chuỗi tính toán nội bộ duy nhất, xuất thẳng sóng âm PCM 16 kHz.

2. Pha 2 – Ghép nối Toàn trình & Kiểm định Liên tầng (Inter-step Chaining):

$$\text{Audio Vào} \xrightarrow{\text{Step 0: Lọc nhiễu}} \text{Step 1: ASR} \xrightarrow{\text{Step 2: Dịch MT}} \text{Step 3: Tổng hợp TTS} \rightarrow \text{Audio Ra}$$

- *Truyền dữ liệu Zero-copy:* Các bước trao đổi dữ liệu bất đồng bộ qua bộ đệm tuần hoàn (*Circular Buffer*) trên RAM LPDDR5 theo mô hình Producer–Consumer, triệt tiêu độ trễ I/O.
- *Cơ chế kiểm soát chất lượng liên tầng:* Tự động hạ chữ thường (`hyp.lower()`) từ Zipformer trước khi nạp vào MT để bảo toàn mã hóa BPE; đồng thời dùng SenseVoice nhận diện ngược âm thanh TTS để tự động sinh lại nếu phát hiện lỗi lặp âm tiết.
- *Ngân sách độ trễ toàn trình (Latency Budget):* Tổng thời gian xử lý toàn chuỗi chỉ tốn 1.0–1.8 giây (ASR ~ 0.2 s, MT ~ 0.6 s, TTS ~ 0.5 s), hoàn toàn nằm dưới giới hạn khắt khe ≤ 3.0 giây của OneVoice AI Challenge.